Scenario C: crew authorization for flight-deck recovery

To: ShipOS systems engineering and mission operations

From: Team 6, ShipOS

Basis: merged `fcd8c58` (PR #46); simulated prototype, 17 October 2026.

Operational rule and authority

The flight deck must be clear and confirmed before recovery starts. Deck machinery must not run while an active mode prohibits it. `shipos/crew/policy.py` evaluates these rules without actuation; `crew/lifecycle.py` retains per-session evidence. The opt-in simulator controller calls these libraries and the public transition planner, then applies only supported chosen engine actions. The API exposes actual gates, pending/applied/blocked tasks and recorded operator confirmation. Observation mode remains available and never applies recommendations to the simulated ship.

A planner's `confirm="human"` is a requirement, not permission received. At 14 or 6 crew, recovery remains blocked until a valid explicit operator input is recorded. The command does not clear an occupied deck. Preparation must finish, and the gate is checked again when the operation starts. Occupation or a crew change invalidates confirmation; a later clear-deck episode requires fresh authorization. Fast-forward never invents a human token. All times below are simulated seconds or planning bounds.

Crew complements and fail-closed behavior

Crew	Deck preparation / evidence	Recovery authorization
14	Upper-bound deck clear 300 s, then human confirmation preparation 60 s.	Actual human confirmation required; occupied or unconfirmed deck blocks.
6	Upper-bound deck clear 720 s, then confirmation preparation 180 s. Early forecast needed.	Actual human confirmation required; late preparation cannot be labelled a successful landing.
0	No crew to clear an occupied deck. A clear deck requires fresh, same-tick simulated evidence.	Automatic only when the actual policy gate permits; missing/stale/obstructed evidence blocks.

Zero-crew controller evidence is derived from simulated deck state and explicitly labelled `simulated_deck_state`; library tests also use labelled simulated camera inputs. There is no camera hardware integration, remote watchkeeper or certified autonomous deck verification. A denial reports engine risk/outcome; no invented diversion site, ditch completion or successful landing is fabricated.

Six-crew forecast requirement

Using declared upper bounds and the G4 assumed 180 s approach:

Crew	Deck plus confirmation lead	Including approach
14	300 + 60 = 360 s (6 min)	540 s (9 min)
6	720 + 180 = 900 s (15 min)	1080 s (18 min)

The declared critical-battery landing window is 720 s (12 min). At 6 crew, reacting only to the critical alert is too late under these bounds. For a forecast deadline at t=1200, `start_by = 1200 - 180 - 900 = 120`: begin by minute 2. This is a planning calculation, not a demonstrated physical flight. Speed/course/noise settle concurrently where the planner permits; acoustic hold completion precedes manoeuvres. Durations must not all be added indiscriminately to the critical path.

`early_clearance()` rejects missing, future-dated or nonfinite forecasts and blocks weather outside card recovery limits. A late result requests immediate preparation but is an operational warning, not proof of arrival. The controller accepts explicit recorded `battery_forecast` events through its public Runner seam; the dashboard battery-report control is not a forecast or an automatic prediction service.

Machinery interlock and runtime evidence

Interlocks follow declared mode conditions, not hardcoded equipment pairs. G3 surveying forbids deck machinery, so G4 recovery/G2 winch preparation must wait for the conflicting state to change. A chosen pause/suspend uses the real declared hold and acoustic-delivery prerequisites; the controller cannot merely label a running machine off. Physical feasibility remains owned by the existing planner/engine.

Merged API acceptance tests run Scenario B under survey and SAR: at t=480 D3 chooses G3 pause/suspend and G4 recovery; hold delivery completes at 540, deck clearing at 780 and preparation at 840. G4 stays flying until explicit confirmation at 841, then recovers; the declared continuation enters launching at 1021 and later flying, **not charging**. The 6/0-crew baseline occupied-deck runs return real denial/blockers. The tests export captured inputs and re-execute byte-identical records. Library unit tests cover early clearance, stale evidence, weather and lifecycle invalidation. Browser tests cover controls/rendering, including some controlled responses; they are not all proof of full physical mission execution.

Remaining boundaries and operator procedure

Run Scenario B before Scenario A. Choose crew/priority before enabling execution; a fresh execution/environment run archives prior evidence. Inspect blockers and pending due times, provide confirmation only after the deck is actually clear, and export the evidence bundle before restarting the in-memory server. Reset archives the current run and restores LIVE A with observation mode and baseline inputs. Replay requires compatible code, callback and assumption registry in addition to captured inputs; the final hash alone is not execution replay.

A generator trip does not imply overload. Baseline observation demand at minute 12 is 49.5 kW. Charging overlaps are separate synthetic inputs. Ambiguous D3 deferrals remain blocked; exact D4 metadata producer #43 is not merged in this baseline. Winner-protection lifetime is provisional and protection may persist indefinitely; the approved internal-resume exception is not a general release policy. G6 is absent. Engineering timing/stability/weather proxies and simulated evidence are not certification. No mentor validation, ship trials or measured latency compliance is claimed.

Sources: crew policy, evidence lifecycle, execution semantics, API acceptance tests. Rendered memo.